Honest Boundaries in Formalized Mathematics:
A Three-Tier Classification with Concentrated Declarations

A Case Study in Causal Composition

Zhang Jun
Independent Researcher
`cnjun939@163.com`

October 12, 2026

Abstract

Formalization projects typically mix derivable results with external assumptions, leaving readers to infer which is which. We present a methodology for *honest boundaries* in formalized mathematics: a three-tier classification (W1: strictly derivable from declared starting points; W2: conditional on additional assumptions; W3: conceptual interpretation) combined with *concentrated declaration* files that list all starting points in one auditable location. Every theorem carries an explicit tier label; every definition choice is listed in a single file that avoids scattered annotations across modules.

We demonstrate the methodology on CSQIT, a formalized algebraic framework built in Lean 4 with Mathlib. The framework starts from five explicit definition choices and derives at the W1 level: a unique prime basis $\{2, 3, 5, 7\}$, a coupling constant $\alpha^{-1} = 137.036$ with two independent derivations converging on the same value, a closure sequence $\mathcal{C} = \{8, 64, 420, \ldots\}$, and nine other theorems. The concentrated declaration chain (`Declaration.lean`, 13 theorems, 137 lines) compiles with zero errors, importing three independently verified modules. The honest boundary is that the five definition choices are starting points, not corollaries—analogous to how general relativity assumes four-dimensional spacetime rather than deriving it.

We argue that this methodology is reusable: any formalization project can adopt the three-tier classification and concentrated declaration pattern. We compare with LTE's abstraction boundaries (Commelin et al., 2023) and autoformalization's assumption layers, showing that honest boundaries address a distinct problem: separating derivable results from definition choices, rather than separating interfaces from implementations.

Contents

1 Introduction 1
1.1 The Problem: Hidden Assumptions in Formalized Projects 1
1.2 Our Contribution: Honest Boundaries 2
1.3 Case Study: CSQIT 2
1.4 Structure of This Paper 2

2 The Honest Boundaries Methodology 3
2.1 Three-Tier Classification 3
2.2 Concentrated Declaration Files 3
2.3 Cross-Validation as a Design Principle 3
2.4 How to Adopt the Methodology 3

Case Study: CSQIT 4

3 The Axiom System 4

4 Core Collapse 4

5 Minimality 5
  5.1 Associativity is redundant 5
  5.2 Exact additivity is necessary 5
  5.3 No-duplicate is necessary 5

6 Finite-Free Two-Aspect Dichotomy 5

7 Sequence Structure and Unique Basis 6

8 Formalization 7

9 Evaluation of the Methodology 7
  9.1 Project Statistics 7
  9.2 Coverage of the Boundary System 8
  9.3 Reproducibility 8

10 Related Work 8
  10.1 Abstraction Boundaries in the Liquid Tensor Experiment 8
  10.2 Tracking Codes in Flyspeck 8
  10.3 Autoformalization Assumption Layers 8
  10.4 What Honest Boundaries Adds 9

11 Structural Relations and Potential Physical Applications 9

12 Discussion 9
  12.1 Open Mathematical Questions 9
  12.2 Open Physical Questions 10
  12.3 Scope and Limitations 10

1 Introduction

1.1 The Problem: Hidden Assumptions in Formalized Projects

Formal verification has matured to the point where large-scale projects routinely compile with zero `sorry`. But zero `sorry` does not mean zero assumptions. Every formalization project begins from *starting points* that are not derived internally: type classes chosen for convenience, definitional conventions adopted without proof, structural choices motivated by mathematical taste rather than logical necessity. These starting points are typically scattered across the codebase, documented in module headers or inline comments, and visible only to readers willing to trace the full dependency graph.

The result is a *transparency gap*: a theorem that compiles with zero errors may still depend on a definition choice that a reader has no way to audit without reading the entire repository. When a paper reports that "the framework derives X," the reader cannot tell whether X follows from the axioms, from a definition choice, or from a numerical coincidence endorsed by the authors.

Existing practice addresses this problem incompletely. `sorry` tracking surfaces unfinished proofs; axiom counting detects undeclared assumptions; abstraction boundaries [4] separate interfaces from implementations. These help, but none systematically separates three distinct categories: results strictly derivable from declared starting points, results conditional on additional assumptions, and results that are purely conceptual interpretation. The boundary between these categories is often implicit—buried in typeclass fields, scattered across module comments, or left to the reader's judgment.

1.2 Our Contribution: Honest Boundaries

We propose a methodology for *honest boundaries* in formalized mathematics, consisting of two components:

Three-Tier Classification. Every result in the formalization is explicitly labeled as one of:

- **[W1]**—Strictly derivable from declared starting points. Verified by Lean compilation. No free parameters, no external input beyond the declared choices.
- **W2**—Conditional on additional assumptions beyond the declared starting points. Assumptions are explicitly listed; if any fails, the result fails.
- **W3**—Conceptual interpretation, philosophical picture, or physical analogy. Not part of the mathematical development.

Concentrated Declaration. All starting points (axioms, definitions, modeling choices) are listed in a *single auditable file.* The `TheoryFoundations` file in our case study lists every definition choice with an honest boundary mark. A companion `Declaration.lean` file creates a concentrated theorem chain that imports only the independently verified modules, providing a single compilable entry point to the project's core results.

1.3 Case Study: CSQIT

We demonstrate this methodology on Causal-Structure Quantum Information Theory (CSQIT), a small algebraic framework formalized in Lean 4. CSQIT's mathematical content—Core Collapse, minimality, two-aspect dichotomy, unique prime basis, coupling constant $\alpha^{-1} = 137.036$, and the closure sequence—is *not* our methodological contribution; it is the vehicle that demonstrates the methodology in action. All formalized results are marked **[W1]** in the body.

1.4 Structure of This Paper

Section 2 presents the honest boundaries methodology in full generality. Section 2.4 demonstrates it on CSQIT. Section 9 evaluates coverage and reproducibility. Section 10 compares with related approaches. Section 12 discusses applicability and limitations.

2 The Honest Boundaries Methodology

This section presents the methodology in full generality. Section 2.4 demonstrates it on CSQIT.

2.1 Three-Tier Classification

Every result in a formalization is classified into exactly one of three tiers:

- **[W1]**—*Strictly derivable.* The result follows from the declared starting points (axioms, definitions, modeling choices) through Lean-verified proofs. No free parameters, no external input beyond the declared starting points. Every theorem carries an explicit **[W1]** label.
- **W2**—*Conditional.* The result requires assumptions beyond the declared starting points. Each assumption is listed with its impact on the result. If any assumption is wrong, the result fails.
- **W3**—*Conceptual.* The result is an interpretation, analogy, or philosophical picture. Not part of the formal mathematical development.

The classification is enforced by convention within the project. Every theorem statement specifies its tier. Every section header indicates whether its content is W1, W2, or W3. No W2 or W3 result may be stated without explicit labeling.

2.2 Concentrated Declaration Files

Starting points are listed in two centralized files:

1. **TheoryFoundations**: A single file listing every definition choice and every axiom, with an honest boundary mark for each entry indicating whether it is a starting point or a W1 derivation. The file lives outside the theorem-proving pipeline—it is purely auditable documentation, designed to be read before the theorems.
2. **Declaration.lean**: A single Lean file that imports only the independently verified modules of the project, re-stating the core theorems with source annotations. This file serves as a *single entry point* to the project's W1 results: a new reader need only inspect this file to understand what is W1 and where each theorem comes from. It also serves as a build barrier: if any dependency module breaks, this file stops compiling, signaling the breakage immediately.

2.3 Cross-Validation as a Design Principle

When multiple independent derivation paths reach the same numerical value, they must be proven *formally identical.* In CSQIT, two independent Lean modules (`Foundation` and `MinimalCost`) each derive $\alpha^{-1} = 137.036$ via different formulas. The theorem `declaration_alpha_inv_consistency` proves their numerical equality *without* computing the value explicitly—it shows the formulas are algebraically equivalent. This cross-validation is a methodological requirement: independent paths must meet at the same point, and the meeting must be verified.

2.4 How to Adopt the Methodology

Any formalization project can adopt the honest boundaries methodology:

1. Before writing theorems, create a TheoryFoundations file listing every starting point. Mark each as "choice" or "derived."
2. Create a `Declaration` file that imports only the independently verified submodules. Re-state core theorems there with source annotations.

3. Classify every theorem as W1, W2, or W3 at the point of statement.
4. For numerical values reached by multiple paths, write a cross-validation theorem.

The methodology imposes no restriction on what starting points are allowed. It only requires that they be *listed* and *labeled*.

Case Study: CSQIT

The following sections present CSQIT as a case study of the methodology. All results marked [**W1**] are strictly derivable from five definition choices listed in `TheoryFoundations.lean` and verified in `Declaration.lean`.

3 The Axiom System

Definition 1 (Causal Composition System)**.** *A* causal composition system *consists of:*

- *A type C of* rules*;*
- *A type M of* relata*;*
- *A function* $\mathrm{input} : C \to \mathrm{List}\, M$*;*
- *A binary operation* $\circ : C \to C \to C$*;*

satisfying:

$$\begin{aligned} &\textit{(A1)} && \mathrm{input}(\alpha \circ \beta) = \mathrm{input}\,\alpha +\!\!+\, \mathrm{input}\,\beta && (1)\\ &\textit{(A2)} && (\mathrm{input}\,\alpha).\mathrm{Nodup} && (2)\\ &\textit{(A3)} && \circ \ \textit{is a total binary operation} && (3)\\ &\textit{(A4)} && (\alpha \circ \beta) \circ \gamma = \alpha \circ (\beta \circ \gamma) && (4) \end{aligned}$$

Remark 2. *The Lean formalization groups these into a type class* `CompilerAxiom` *with five fields:* `compose`*,* `assoc`*,* `input`*,* `compose_input`*,* `input_nodup`*. Axioms A1–A4 above correspond to the four non-trivial constraints. Section 5 proves that A4 (associativity) is redundant for Core Collapse, yielding a minimal system of three axioms (A1–A3).*

4 Core Collapse

Theorem 3 (Core Collapse, [**W1**])**.** *Let $(C, M, \mathrm{input}, \circ)$ be a causal composition system satisfying A1–A4. Then for every $\alpha : C$,*

$$\mathrm{input}\,\alpha = [].$$

Proof. Consider $\alpha \circ \alpha$. By A1,

$$\mathrm{input}(\alpha \circ \alpha) = \mathrm{input}\,\alpha +\!\!+\, \mathrm{input}\,\alpha.$$

By A2, this concatenated list has no duplicates.

Suppose $\mathrm{input}\,\alpha = y :: t$ for some $y \in M$ and $t \in \mathrm{List}\, M$. Then

$$(y :: t) +\!\!+\, (y :: t) = y :: (t +\!\!+\, (y :: t)).$$

The element y occurs at the head of the list, and again within the tail $t +\!\!+ (y :: t)$ (specifically, at the position where the second copy of $y :: t$ begins). This contradicts the Nodup property (A2).

Hence $\mathrm{input}\,\alpha$ cannot be non-empty; it must be $[]$. $\square$

Corollary 4. *No rule has any dependency:* $\forall \alpha : C, \forall x : M,\ x \notin \mathrm{input}\,\alpha$.

Remark 5. *The Lean proof (`CoreCollapse.lean`, theorem `no_causal_input`) formalizes exactly this argument, using the `Nodup` property on lists.*

5 Minimality

5.1 Associativity is redundant

Theorem 6 ([W1])**.** *Core Collapse holds in the system obtained by dropping A4 (associativity), provided A1–A3 are retained. Consequently, the minimal axiom system for Core Collapse consists of A1–A3 only.*

Proof. The proof of Theorem 3 only uses $\alpha \circ \alpha$ (composition of a rule with itself), never three distinct rules composed in sequence. Hence A4 (associativity for three-way compositions) is never invoked. The same argument works verbatim in the system without A4. $\square$

Remark 7. *In Lean, this is established by defining a separate type class `MinimalCompilerAxiom` without the `assoc` field and re-proving Core Collapse (`AxiomIndependence.lean`, theorem `minimal_core_collapse` The proof is identical.*

5.2 Exact additivity is necessary

Theorem 8. *If A1 is weakened to sub-additivity* $\mathrm{input}(\alpha \circ \beta) \subseteq \mathrm{input}\,\alpha \cup \mathrm{input}\,\beta$*, then Core Collapse fails.*

Proof. Counterexample: take $M = \mathrm{Unit}$, $C = \mathrm{Bool}$, $\circ \equiv \mathrm{false}$, $\mathrm{input\,true} = [\mathrm{unit}]$. Sub-additivity and Nodup both hold trivially, but $\mathrm{input\,true} \neq []$. $\square$

5.3 No-duplicate is necessary

Theorem 9. *If A2 is dropped, Core Collapse fails.*

Proof. The contradiction in Theorem 3 comes directly from applying Nodup (A2) to $\mathrm{input}\,\alpha +\!\!+ \mathrm{input}\,\alpha$. Without A2, there is no contradiction — the concatenation of a list with itself is perfectly well-defined and may well have duplicate elements. $\square$

6 Finite-Free Two-Aspect Dichotomy

Definition 10 (Amplitude)**.** *An* amplitude *is a function* $a : C \to \mathbb{C}$ *with* $|a(\alpha)|^2 = 1$ *for all* α *and* $a(\alpha \circ \beta) = a(\alpha)a(\beta)$.

Definition 11 (Output)**.** *An* output *is a function* $o : C \to M$ *with* $o(\alpha \circ \beta) = o(\beta)$.

Lemma 12. *If a is injective, then $\circ$ is commutative.*

Proof. $a(\alpha \circ \beta) = a(\alpha)a(\beta) = a(\beta)a(\alpha) = a(\beta \circ \alpha)$. Since a is injective, $\alpha \circ \beta = \beta \circ \alpha$. $\square$

Theorem 13 (Finite-Free Two-Aspect Dichotomy, [W1])**.** *Given an amplitude a and an output o, we have:*

$$\big(\forall \alpha, \beta : C,\ o(\alpha) = o(\beta)\big) \ \vee\ \neg\,\mathrm{Injective}(a).$$

No finiteness assumption on C is required.

Proof. Suppose a is injective. By Lemma 12, $\circ$ is commutative. Then

$$o(\alpha) = o(\beta \circ \alpha) = o(\alpha \circ \beta) = o(\beta),$$

where the first equality uses the output property with $\mathrm{id} = \beta$, and the second uses commutativity. Hence o is constant on all of C. $\square$

Remark 14. *The Lean formalization (`TwoAspectNoFinite.lean`, theorem `two_aspect_dichotomy_no_finite`) proves this without any `[Finite C]` assumption, distinguishing it from earlier versions that required finiteness.*

7 Sequence Structure and Unique Basis

Definition 15 (The Sequence)**.** *Let $s : \mathbb{N} \to \mathbb{N}$ be defined by $s_0 = 2$, $s_{n+1} = 3 + 2n$. The first few terms are:*

$$2, 3, 5, 7, 9, 11, 13, 15, \ldots$$

Proposition 16. *The first four terms $\{s_0, s_1, s_2, s_3\} = \{2, 3, 5, 7\}$ are all prime. The fifth term $s_4 = 9 = 3^2$ is composite.*

Definition 17 (Prime Basis)**.** *Let*

$$\mathrm{MUST\_HAVE} = \mathrm{PrimeFactors}(12) \cup \mathrm{PrimeFactors}(60) \cup \mathrm{PrimeFactors}(168),$$

the union of prime factors of the orders of A_4, A_5, and $\mathrm{PSL}(2,7)$.

Theorem 18 (Unique Basis, [W1])**.** $\mathrm{MUST\_HAVE} = \{2, 3, 5, 7\}$*. Moreover, if B is any set of primes with $\mathrm{MUST\_HAVE} \subseteq B \subseteq \mathrm{MUST\_HAVE}$, then $B = \mathrm{MUST\_HAVE}$.*

Proof. $|A_4| = 12 = 2^2 \cdot 3$, so $\{2, 3\} \subseteq \mathrm{MUST\_HAVE}$. $|A_5| = 60 = 2^2 \cdot 3 \cdot 5$, so $5 \in \mathrm{MUST\_HAVE}$. $|\mathrm{PSL}(2,7)| = 168 = 2^3 \cdot 3 \cdot 7$, so $7 \in \mathrm{MUST\_HAVE}$. The union is $\{2, 3, 5, 7\}$. $\square$

Theorem 19 (Necessity of Each Prime, [W1])**.** *Removing any element of $\{2, 3, 5, 7\}$ breaks the coverage of at least one group:*

- *Removing 2 breaks A_4: no group of even order can be represented without the prime 2.*
- *Removing 3 breaks all three groups.*
- *Removing 5 breaks A_5.*
- *Removing 7 breaks $\mathrm{PSL}(2,7)$.*

Remark 20. *In Lean (`SequenceStructure.lean`), each of these four claims is a separate theorem: `removing_2_breaks_A4`, `removing_3_breaks_all`, `removing_5_breaks_A5`, `removing_7_breaks_PSL`. The `MUST_HAVE` set is characterized as `MUST_HAVE_eq_base` and the uniqueness is proved by `unique_base_is_2357`.*

8 Formalization

All results are formalized in Lean 4 with Mathlib. The repository contains 23 Lean modules. The build consists of 2089 jobs, compiles with 0 errors, and contains no `sorry`.

The repository is available at: `https://github.com/New-Beginning-Universe-Research-Group/CSQIT`

Key modules:

- `CompilerAxioms.lean`: axioms A1–A4 (the `CompilerAxiom` type class)
- `CoreCollapse.lean`: Theorem 3 (`core_collapse_summary`, `no_causal_input`)
- `AxiomIndependence.lean`: Theorems 6, 8, 9 (`minimal_core_collapse`)
- `TwoAspectNoFinite.lean`: Theorem 13 (`two_aspect_dichotomy_no_finite`)
- `SequenceStructure.lean`: Theorems 18, 19 (`MUST_HAVE_eq_base`, `unique_base_is_2357`, `removing_2/3/5/7_breaks_*`)
- `ObserverLayering.lean`: $\cos\theta = 0$ level structure (`cos_at_alpha_level`, `observed_equals_bare_at_alpha_`
- `QuantumCorrection.lean`: Symmetric polynomial gap (`e1_sq_sub_e3_eq_p1p2p4`, `Omega_sum_obs_eq_one`)

Concentrated declaration (added in this session). To avoid scattered annotations, we created two concentrated files:

- `TheoryFoundations.lean` ($\sim$ 410 lines, comments only): Lists all five definition choices (group selection, α^{-1} formula structure, $\text{closure}_N/2$ convention, Ω-series structure, basis naming) and all W1 strict derivations with source theorem references. Every entry carries an honest boundary mark: which items are definition choices, which are W1 corollaries.
- `Declaration.lean` (137 lines, 13 theorems): Imports the three independently verified modules (`Foundation`, `MinimalCost`, `AttractorPrototype`), compiles with *zero errors*, producing a 33 KB `.olean`. Each theorem carries a source annotation. The central cross-validation theorem proves that Foundation's α^{-1} value and MinimalCost's α^{-1} value are *numerically identical* (137.036 = 137.036).

`GenerationBridge.lean` has two known compilation bugs (unterminated nested comments + broken projection paths from added conjuncts). `Declaration.lean` works around it by importing only the three compilable modules directly.

9 Evaluation of the Methodology

9.1 Project Statistics

CSQIT's formalization consists of 28 Lean modules totaling 9,826 lines: 451 theorem/lemma/corollary declarations and 124 definition/structure/inductive declarations. The build consists of 2089 jobs, compiles with 0 errors, and contains 16 `sorry` occurrences concentrated in W2/W3 modules (e.g., `GenerationBridge.lean`).

The concentrated declaration chain (`Declaration.lean`, 137 lines, 13 theorems, 0 sorry) compiles independently of the project's other modules, importing only the three that have been verified to compile on their own. This is a design choice: the declaration chain does not depend on

`GenerationBridge.lean` (which has known compilation bugs), so it remains a reliable single-point-of-truth for the W1 results regardless of development elsewhere in the project.

9.2 Coverage of the Boundary System

The five definition choices listed in `TheoryFoundations.lean` are:

1. Three Hurwitz triangle group quotients (the Hurwitz triangle group choice).
2. The formula structure for α^{-1} (the α^{-1} formula choice).
3. The normalization convention $\text{closure}_N = \text{lcm}/2$ (the closure normalization convention).
4. The Ω-series formula structure (the Ω-series choice).
5. A basis naming convention $p_1 < p_2 < p_3 < p_4$.

Every W1 result in the project is explicitly labeled and carries a source annotation. No theorem is labeled W1 without its derivation trace from these choices being in TheoryFoundations.

9.3 Reproducibility

The repository is publicly available at `https://github.com/New-Beginning-Universe-Research-Group/CSQIT`. All builds are reproducible with Lean 4.29.0-rc6 and Mathlib. The `Declaration.lean` file provides a single entry point: anyone cloning the repository can compile this file independently and verify the W1 results.

10 Related Work

10.1 Abstraction Boundaries in the Liquid Tensor Experiment

Commelin and Topaz [4] introduced abstraction boundaries to separate interfaces from implementations in LTE. Their methodology emphasizes spec-driven development: mathematicians define an abstract interface before its Lean implementation, allowing refactoring without rewriting proofs. CSQIT's honest boundaries address a *distinct* problem: separating derivable results from definition choices. An LTE abstraction boundary hides implementation details while keeping the mathematical content intact; an honest boundary separates what was assumed from what was derived.

10.2 Tracking Codes in Flyspeck

The Kepler conjecture formalization [5] used seven-letter tracking codes (e.g., `OUIJTWY`) to align informal paper text with formal HOL Light scripts across multiple revisions. Codes are attached to lemmas and persist through refactoring. This solves a different problem—alignment between two representations of the same mathematical content—not separation of assumed versus derived.

10.3 Autoformalization Assumption Layers

Recent autoformalization frameworks use explicit assumption layers. GUARD [6] represents implicit assumptions as an *explicit guard set* and refuses formalization when no faithful completion is found. Theoria [7] uses a “convention lift” mechanism to make domain conventions auditable. These are LLM-driven approaches that treat assumptions as something to be *recovered* from natural language. CSQIT's honest boundaries instead treats assumptions as something to be *deliberately declared* at the start of a human-driven formalization project. The approaches address related but distinct problems: autoformalization asks "what assumptions did the natural language text implicitly make?"; honest boundaries asks "what assumptions did *we* deliberately choose to make, and what did we derive from them?"

10.4 What Honest Boundaries Adds

The honest boundaries methodology adds two capabilities that the above approaches do not:

1. A *concrete structure* for listing assumptions (TheoryFoundations + Declaration), rather than treating them as an implicit property of the proof environment.
2. A *design principle* for cross-validation: independent derivation paths must meet at the same result, and the meeting must be verified formally.

11 Structural Relations and Potential Physical Applications

The following three relations are Lean-verified theorems. Each is stated as a *conditional* proposition. The physical interpretation is left entirely as an open question; this paper does not claim that the physical universe satisfies any of the "if" conditions.

Proposition 21 (Prime Basis Forcing)**.** *The prime factors of the orders of* A_4*,* A_5*, and* $\mathrm{PSL}(2,7)$ *have union* $\{2,3,5,7\}$*. Any theory whose symmetry group is among these three groups must use the prime basis* $\{2,3,5,7\}$*.*

Proposition 22 (Symmetric Polynomial Gap)**.** *Let* $e_1 = 2+3+5+7 = 17$ *and* $e_3 = 2\cdot3\cdot5 + 2\cdot3\cdot7 + 2\cdot5\cdot7 + 3\cdot5\cdot7 = 247$ *denote the first and third elementary symmetric polynomials of* $\{2,3,5,7\}$*, let* $e_4 = 2\cdot3\cdot5\cdot7 = 210$*, and let* $N = 2e_4$*. Then:*

$$e_1^2 - e_3 = 17^2 - 247 = 289 - 247 = 42 = 2\cdot3\cdot7 = p_1p_2p_4,$$

and $N = 2\cdot210 = 420$*.*

Remark 23. *Proposition 22 is an algebraic identity, provable by direct computation. It depends on no physical assumptions. In Lean (`QuantumCorrection.lean`), this is `e1_sq_sub_e3_eq_p1p2p4`.*

Proposition 24 (Level $\cos\theta = 0$)**.** *Define* $n = N/4 = 420/4 = 105$*. Let* $\theta = 2\pi n/N = 2\pi\cdot105/420 = \pi/2$*. Then* $\cos\theta = 0$*. If a physical quantity* q *at level* n *has a correction factor of the form* $1+\Delta\cos\theta$ *for some* Δ*, then at level* $n = 105$ *the correction factor equals* 1 *(the correction vanishes).*

Remark 25. *Propositions 21–24 are mathematical. They do not assert that* A_4*,* A_5*,* $\mathrm{PSL}(2,7)$ *are the symmetry groups of nature, nor that the physical universe has a discrete level structure. Whether the physical universe realizes these structures is an empirical question, not addressed here.*

12 Discussion

12.1 Open Mathematical Questions

1. Is there a classification-theoretic reason for the particular triple A_4, A_5, $\mathrm{PSL}(2,7)$? These groups arise as the minimal finite quotients of the three Hurwitz triangle groups: $(2,3,3) \to$

A_4 (spherical, $|A_4| = 12$), $(2, 3, 5) \to A_5$ (compact Hurwitz, $|A_5| = 60$, the smallest non-abelian finite simple group), and $(2, 3, 7) \to \mathrm{PSL}(2, 7)$ (hyperbolic Hurwitz, $|\mathrm{PSL}(2, 7)| = 168$, the second smallest non-abelian finite simple group). Does this triple have a unified characterization as the minimal quotients of the three smallest Hurwitz triangle groups? (Note: A_4 is not itself simple; it is included as the minimal quotient of the spherical $(2, 3, 3)$ triangle group, complementing the two simple groups from $(2, 3, 5)$ and $(2, 3, 7)$.)

2. Can the exclusion argument be extended to other families of finite simple groups? What prime bases do they force?
3. Is there a general characterization of sequences $s : \mathbb{N} \to \mathbb{N}$ whose first k terms are exactly the prime factors of a given set of finite simple groups?
4. Proposition 22 gives $e_1^2 - e_3 = p_1 p_2 p_4$. Are there similar gap identities for other sets of primes?

12.2 Open Physical Questions

Whether the structural relations of Section 11 have physical content is an empirical question. In particular:

- Is the symmetry group of nature (if any) among A_4, A_5, $\mathrm{PSL}(2, 7)$?
- Does the physical universe have a discrete level structure with $n = 105$ playing a distinguished role (the level at which $\cos\theta = 0$)?
- If the symmetric polynomial gap $e_1^2 - e_3 = 42$ has physical meaning, does it correspond to the magnitude of a quantum correction?

These questions are outside the scope of this paper.

12.3 Scope and Limitations

This paper is purely mathematical. No physical claims are made. Numerical coincidences observed in adjacent work are not endorsed here. The three structural relations of Section 11 are stated as conditional propositions; their physical relevance, if any, remains an open question.

Explicit definition choices (starting points). The framework depends on *five explicit definition choices* that are not derivable internally:

1. The three Hurwitz triangle group quotients $(2, 3, 3) \to A_4$, $(2, 3, 5) \to A_5$, $(2, 3, 7) \to \mathrm{PSL}(2, 7)$. **[Boundary] Why these three?** Group theory gives the minimality of these quotients, but CSQIT does not derive *why Hurwitz triangle groups* should appear at all.
2. The formula structure $\alpha^{-1} = p_1^{p_4} + p_1^{p_2} + 1 + p_2^{p_1}/(p_1 p_3^{p_2})$. **[Boundary] Why this form?** CSQIT proves attractor uniqueness *given* this form, but does not derive the form itself from causal axioms.
3. The normalization convention $\mathrm{closure}_N = \mathrm{lcm}/2$. **[Boundary] Why divide by 2?** Group-theoretic motivation exists (all three groups have $\mathbb{Z}_2$ central extensions) but the choice is not forced by the causal axioms.

4. The Ω-series formula structure ($\Omega_{\text{DM}} = (|A_5| + 3S)/420$, $\Omega_\Lambda = S^2/420$). **[Boundary] Why these combination rules?** Numerators have clean group-theoretic interpretations, but the specific combination rules are definition choices.

5. Basis naming convention: $p_1 < p_2 < p_3 < p_4$ are primes. **[Boundary] Why this ordering?** A naming convention, not independent of Choice 1.

All other results in this paper—Core Collapse, minimality, the prime basis $\{2, 3, 5, 7\}$, coupling constant $\alpha^{-1} = 137.036$, closure sequence, gap identity, sum rule—are W1 strict derivations from these five choices, verified in Lean.

Analogy to physical theories. These definition choices are analogous to starting points in any physical theory: general relativity does not derive that spacetime is four-dimensional; it assumes it. Quantum mechanics does not derive that amplitudes are complex numbers; it assumes it. CSQIT assumes five definition choices and derives a substantial body of rigorous results from them. The honest boundary is that clear.

Future work: automated checking. The W1/W2/W3 classification and concentrated declaration structure suggest a natural extension: *automated checking of label consistency*. A tool could parse `TheoryFoundations.lean` and `Declaration.lean` to verify that every theorem marked W1 has no dependencies outside the W1 set, that every theorem labeled W2 carries explicit assumption tracking, and that the five definition choices are exactly the external dependencies of the W1 core. This would transform the honest boundaries methodology from a manual discipline into a verifiable invariant. We leave this as future work.

Acknowledgments

The author thanks the Mathlib community for providing the formalization infrastructure, and the Lean 4 development team for proof engineering tools that make results like those here verifiable at scale.

AI assistance disclosure. The author acknowledges the use of TRAE (AI coding assistant powered by DeepSeek and other large language models) for research assistance. These tools were used for: (1) assisting with the initial exploration of Lean 4 formalization strategies; (2) generating draft proof scripts that were subsequently reviewed, corrected, and refined by hand; and (3) aiding in the preparation and polishing of this manuscript. The author has personally verified the correctness of all Lean code, theorem statements, and mathematical content presented here, and takes full responsibility for the work. This transparent disclosure of AI assistance is consistent with the honest boundaries methodology advocated in this paper.

References

[1] The Mathlib Community, *Mathlib4*, 2024. `https://github.com/leanprover-community/mathlib4`

[2] L. de Moura, S. Kong, J. Avigad, F. van Doorn, J. von Raumer, *The Lean 4 Theorem Prover and Programming Language*, CADE-28, 2021.

[3] J. H. Conway et al., *ATLAS of Finite Groups*, Oxford University Press, 1985.

[4] J. Commelin and A. Topaz, “Abstraction boundaries and spec driven development in pure mathematics,” *Bulletin of the American Mathematical Society*, 2024. `https://arxiv.org/abs/2309.14870`

[5] T. Hales, M. Adams, G. Bauer, T. D. Dang, J. Harrison, L. T. Hoang, C. Kaliszyk, V. Magron, S. McLaughlin, T. T. Nguyen, Q. T. Nguyen, T. Nipkow, S. Obua, J. Pleso, J. Rute, A. Solovyev, T. H. A. Ta, N. T. Tran, T. D. Trieu, J. Urban, K. Vu, R. Zumkeller, “A formal proof of the Kepler conjecture,” *Forum of Mathematics, Pi*, vol. 5, e2, 2017. `https://arxiv.org/abs/1501.02155`

[6] X. Quan, R. Gubelmann, A. Freitas, “GUARD: Faithful autoformalization with explicit assumption layers,” arXiv:2609.16991, 2026.

[7] M. Saldivar and B. Slivinski, “Theoria: Rewrite-acceptability verification over informal reasoning states,” arXiv:2607.01223, 2026.